Security Assurance for Modern Big Data APIs

Enea Manzi[1], Marco Anisetti[1,2]

enea.manzi@unimi.it
https://eneamanzi.github.io/

[1]Department of Computer Science, Università degli Studi di Milano

[2]Moon Cloud srl

ITADATA 2026

Context: Big Data APIs & Security Assurance

- **Big Data Ecosystem:** distributed services organized into pipelines and exposed through REST APIs
- **Trust Boundary:** API Gateways centralize main security controls.
- **Assurance Gap:** infrastructure and functional testing leave API security semantics largely unchecked
- **Consequence:** An API-level weakness can expose the pipelines and data behind the interface.

State of the Art Limitations and Objectives

State of the Art Limitations

- **Syntactic Blindness:**
  Checks request/response syntax, but not security semantics
- **The Oracle Problem:**
  No automated criterion to evaluate security guarantees
- **Unreliable Execution:**
  Flaky results make continuous CI/CD integration unreliable

Our Objectives

- **Semantic Verification:**
  Verify security behavior beyond request/response syntax
- **Specified Oracles:**
  Define explicit rules for evaluating security guarantees
- **Agnostic & Deterministic:**
  Build a reproducible engine, portable across any documented target.

Our Approach: Contract-Driven API Assurance

Core Idea: Derive valid probes from OpenAPI contracts and evaluate security behavior using specified, deterministic oracles.

- **Application Agnosticism & Contract-Driven Probing**
  OpenAPI and configuration drive the assessment.
- **Specified Oracles for Security Guarantees**
  Explicit rules evaluate each security guarantee.
- **Eight-Domain Taxonomy & Privilege Gradient**
  Security checks are organized by domain and access level.
- **Deterministic Assessment Engine**
  Test execution is coordinated for reproducible results.

Approach Details: Application Agnosticism and Contract-Driven Probing

- **Application Agnosticism:** No target-specific code. The target is described at runtime by its OpenAPI specification and configuration.
- **Configuration as Source of Variation:** Deployment-specific values and operational parameters are kept outside the assessment logic.
- **Contract-Driven Probing:** The specification defines the documented attack surface and guides the construction of valid requests.

Approach Details: Taxonomy and Privilege-Aware Coverage

- **Eight-Domain Taxonomy (D0–D7):** Security guarantees are organized into semantically distinct domains.
- **Privilege-Aware Coverage:** Each test declares the access level it requires:
  - *BLACK_BOX:* Network access only.
  - *GREY_BOX:* Credentials for distinct roles.
  - *WHITE_BOX:* Access to the gateway configuration plane.

Approach Details: Specified Oracles

- **Semantic Evaluation:** Each test evaluates observed behavior against a specific security guarantee.
- **3-Step Oracle Construction:**
  1. Identify the guarantee under verification.
  2. Define what a violation looks like.
  3. Encode the resulting evaluation rule.
- **Epistemic Limit:** Verdicts are only as sound as the contract they are checked against.

Approach Details: Deterministic Assessment Engine

- **Dependency-Aware Scheduling:** Resolves test dependencies into a deterministic execution order.
- **Reproducible Assessment:** Fixed configuration, ordering, and oracles make relevant results comparable across runs.
- **Actionable Output:** Structured results and semantic exit codes support automated CI/CD decisions.

Experimental Setup

Target and gateway

- **Forgejo** 14.0.3 – **Kong** 3.9 DB-less
- PostgreSQL 15 as the persistence backend

Target properties

- Native OpenAPI specification
- Distinct privilege levels and parameterized endpoints
- Gateway configuration available for white-box checks

The testbed combines expected successes, deliberately broken controls, and native target behavior.

Experimental Results

Two independent runs

produced the same outcomes across **18 active tests in 7 domains**

9 PASS **7 FAIL** **2 SKIP** **0 ERROR**

Key result: A Discrepancy Between Contract and Behaviour

Forgejo's OpenAPI specification declares the endpoints as protected, but the service accepts requests to them without credentials.

The specified oracle reports a discrepancy and classifies the behaviour as BYPASSED.

Conclusions & Future Work

Conclusions

- Contract-driven assessment verifies security behavior beyond syntactic conformance.
- Specified oracles and the eight-domain taxonomy organize semantic security checks.
- The deterministic engine distinguishes enforced from violated guarantees and supports reproducible continuous verification.

Future Work

- Complete the coverage of the taxonomy and add adapters for managed cloud gateways.
- Develop reliable methods to generate security oracles automatically.

Thanks! Questions?

Enea Manzi
enea.manzi@unimi.it
https://eneamanzi.github.io/
https://www.linkedin.com/in/eneamanzi